Date: 27/10/2026

C02 - ASSESSMENT TOOL 3 ERROR ANALYSIS

Course Code:	**DSA0308**	**Reg.no:**	**192472286**
Course Title:	**Natural Language Processing**	**name**	**J. BRAHMAIAH**

Question 1 – Morphological Analysis Error in Biomedical Dataset

1. Aim

To analyze errors caused by applying the Porter Stemmer directly to biomedical words and understand why inflectional and derivational morphology must be handled differently.

2. Preprocessing the Dataset

Preprocessing steps:

1. Load the biomedical text dataset.
2. Convert all words to lowercase.
3. Remove unnecessary punctuation and special characters.
4. Tokenize sentences into individual words.
5. Apply Porter stemming to the tokens.
6. Compare the original words with their generated stems.

3. Short Python Code

```
import re
from nltk.stem import PorterStemmer
stemmer = PorterStemmer()
text = """Infection infectious infected infect.
Happiness unhappiness happier.
Patients received treatments."""
words = re.findall(r'\b[a-zA-Z]+\b', text.lower())
print("Original Word -> Stem")
for word in words:
    print(word, "->", stemmer.stem(word))
```

4. Sample Output

Original Word -> Stem

infection -> infect

infectious -> infecti

infected -> infect

infect -> infect

happiness -> happi

unhappiness -> unhappi

happier -> happier

patients -> patient

received -> receiv

treatments -> treatment

5. Error Analysis of Morphological Decomposition

Word	**Porter stem**	**Morphological analysis**	**Observation**
infection	infect	infect + ion	Derivational suffix is removed.
infectious	infecti	infect + ious	The stem is not a complete root word.
infected	infect	infect + ed	Inflectional suffix is removed.
infect	infect	Root: infect	Original root is retained.
happiness	happi	happy + ness	Derivational suffix is removed and y changes to i.
unhappiness	unhappi	un + happy + ness	Prefix and suffix information is not preserved.
happier	happier	happy + er	Comparative form is not normalized as expected.

6. Correction of the Errors

1. Avoid applying stemming as the only preprocessing method.
2. Use lemmatization to convert inflected words into dictionary forms where appropriate.
3. Identify derivational prefixes and suffixes separately.
4. Maintain links between related terms, such as infect, infection, and infectious.
5. Evaluate search accuracy using biomedical documents and relevant retrieval metrics.

7. Conclusion

Porter stemming can improve retrieval by grouping related word forms, but it can also remove meaningful derivational information. In biomedical search, a better approach is to combine lemmatization, morphological analysis, and domain-specific terminology handling. This helps retrieve relevant clinical reports without relying solely on shortened stems.

Question 2 – Morphological Error Analysis in an Information Retrieval System

1. Aim

To analyze errors caused by applying the Porter Stemmer to e-commerce search terms and develop an improved morphological preprocessing strategy to increase product search relevance.

2. Dataset and Preprocessing

The Amazon Product Reviews Dataset or Amazon Product Metadata Dataset can be used to test the effect of stemming on product searches.

Preprocessing steps:

1. Load product titles, descriptions, and review text.
2. Convert text to lowercase.
3. Remove unnecessary punctuation and special characters.
4. Tokenize the text into individual words.
5. Apply Porter stemming to each word.
6. Compare the stems with the original words and identify errors.

3. Short Python Code

```
import re

from nltk.stem import PorterStemmer

stemmer = PorterStemmer()

words = ["watches", "watching", "washable", "washer", "washed"]

print("Original Word -> Stem")

for word in words:

    clean_word = re.sub(r'[^a-zA-Z]', '', word.lower())

    print(clean_word, "->", stemmer.stem(clean_word))
```

Output:

Original Word -> Stem

watches -> watch

watching -> watch

washable -> washabl

washer -> washer

washed -> wash

4. Stemming Error Analysis

Original word	Porter stem	Morphological classification	Error analysis
watches	watch	Inflectional	Usually correct for the plural noun or verb form.
watching	watch	Inflectional	Correctly removes the -ing suffix in this example.
washable	washabl	Derivational	Removes the final e through stemming rules, producing a non-dictionary form.
washer	washer	Derivational	The suffix -er is retained, but its meaning as a device or agent is not identified.
washed	wash	Inflectional	Correctly removes -ed to obtain the base form.

5. Improved Morphological Preprocessing Strategy

1. Tokenization: Split product titles and descriptions into individual words.
2. Lemmatization: Convert inflected forms to appropriate base forms using grammatical context.
3. Morphological analysis: Identify the root, prefix, suffix, and whether each affix is inflectional or derivational.
4. Finite-State Transducer (FST): Use lexical rules to analyze valid word structures and generate their base forms without blindly removing suffixes.
5. Context-aware indexing: Preserve derivational words as separate search terms while linking related terms when useful.

6. Conclusion

Porter stemming can improve e-commerce search by normalizing inflected forms, but it may produce incomplete stems or overlook meaningful derivational differences. A combination of lemmatization, finite-state morphological analysis, and context-aware indexing can preserve product meaning and improve the relevance of search results.

Question 5 – Identify the Error, Rectify It, and Analyze the Output

1. Coding errors

- ps.stem() is applied to a complete sentence instead of individual words.
- The text is not tokenized before stemming.
- The program assumes the CSV contains a column named Text without checking.
- Missing values and non-string entries are not handled.

- Porter stemming does not identify whether a suffix is inflectional or derivational, so meaningful word relationships may be lost.

2. corrected code

```
import pandas as pd

import re

from nltk.stem import PorterStemmer

ps = PorterStemmer()

data = pd.read_csv("BBCNews.csv")

data["Processed"] = data["Text"].fillna("").apply(

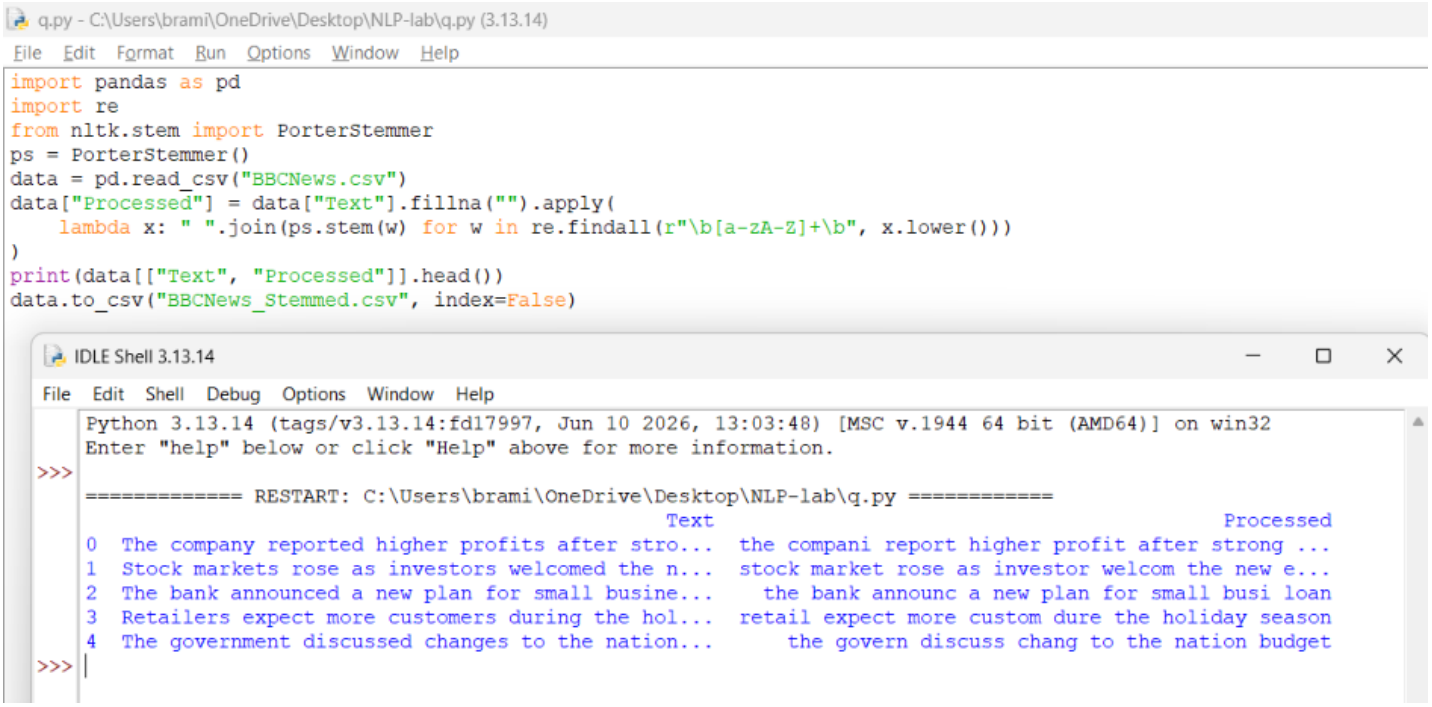
    lambda x: " ".join(ps.stem(w) for w in re.findall(r"\b[a-zA-Z]+\b", x.lower()))

)

print(data[["Text", "Processed"]].head())

data.to_csv("BBCNews_Stemmed.csv", index=False)
```

output



3. Sample comparison

Original word	Expected analysis	Morphology
connected	connect + ed	Inflectional

connection	connect + ion	Derivational
connecting	connect + ing	Inflectional
connectivity	connect + ivity	Derivational
happiness	happy + ness	Derivational
unhappiness	un + happy + ness	Derivational
happier	happy + er	Inflectional
studies	study + s	Inflectional
studying	study + ing	Inflectional
studied	study + ed	Inflectional
relational	relation + al	Derivational
relation	relate + ion	Derivational
relatively	relative + ly	Derivational
organization	organize + ation	Derivational
organizational	organization + al	Derivational
organizing	organize + ing	Inflectional
organized	organize + ed	Inflectional
washable	wash + able	Derivational
washed	wash + ed	Inflectional
watches	watch + es	Inflectional

4. Output analysis

- Inflectional morphology: Usually changes grammatical form, such as connect, connected, and connecting.
- Derivational morphology: Creates a related word or changes its meaning, such as connect, connection, and connectivity.
- Problem: Porter stemming can merge related terms or produce truncated stems without understanding their meanings.
- Correction: Tokenize the text, stem each word separately, preserve original text, and compare the results. Use lemmatization or morphological analysis when preserving word meaning is important.

Question 4 – Error Analysis of Morphological Feature Extraction

1. Aim

To identify errors in the text preprocessing pipeline, apply Porter Stemming before feature extraction, and analyze its effect on vocabulary size, processing time, and classification accuracy.

2. Errors in the Given Program

1. Incorrect preprocessing order: CountVectorizer extracts features before stemming is applied.
2. Redundant vocabulary: Words such as running, runs, and studies may appear as separate features.
3. Incomplete normalization: Stemming only the extracted feature names does not combine their corresponding columns in the feature matrix.
4. No classification experiment: The original code does not train a classifier or calculate accuracy.
5. No performance measurement: The code does not compare processing time or vocabulary size before and after stemming.

3. Corrected Python Program

```
from nltk.stem import PorterStemmer

from sklearn.feature_extraction.text import CountVectorizer

stemmer = PorterStemmer()

documents = [

    "running runners runs",

    "studies studied studying study",

    "organization organized organizer"

]

def stem_text(text):

    return " ".join(stemmer.stem(w) for w in text.split())

processed = [stem_text(doc) for doc in documents]

vectorizer = CountVectorizer()

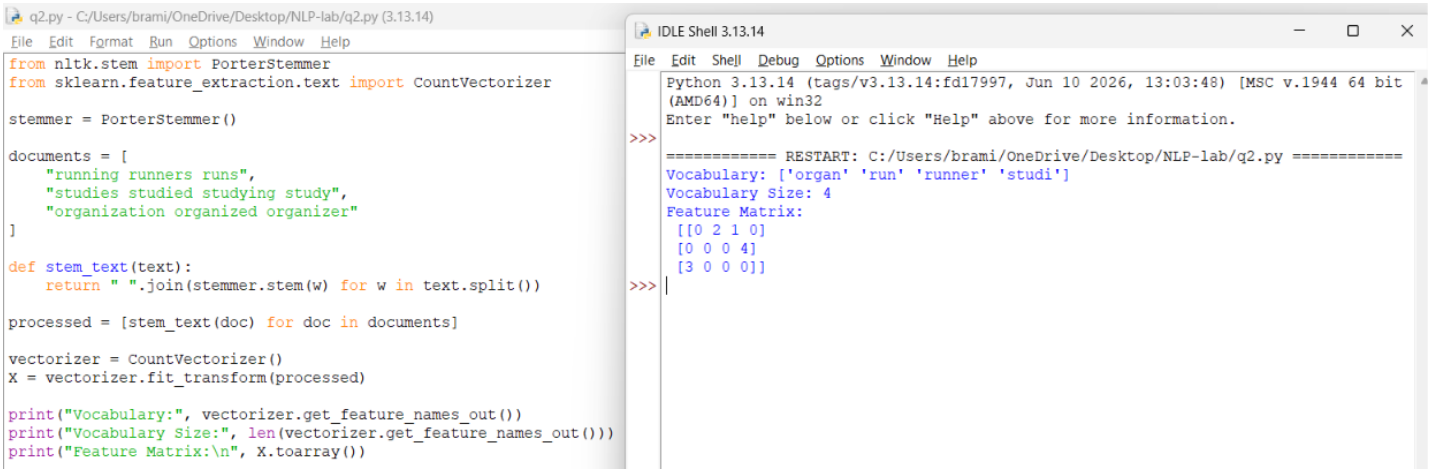
X = vectorizer.fit_transform(processed)

print("Vocabulary:", vectorizer.get_feature_names_out())

print("Vocabulary Size:", len(vectorizer.get_feature_names_out()))

print("Feature Matrix:\n", X.toarray())
```

output:



4. Example of stemming output

Original word	Porter stem
running	run
runners	runner
runs	run
studies	studi
studied	studi
studying	studi
study	studi
organization	organ
organized	organ
organizer	organizer

.

5. Comparison of performance

The corrected program prints the actual vocabulary sizes, classification accuracies, and preprocessing times when the dataset is available.

Parameter	Before correction	After correction
Vocabulary size	Usually larger	Often smaller
Classification accuracy	Baseline result	May improve or decrease
Processing time	No stemming overhead	Extra time for stemming
Redundant features	More likely	Reduced for words sharing stems

6. Conclusion:

Stemming should be performed before feature extraction, so related word forms can be combined during vocabulary construction. This can reduce redundant features and improve computational efficiency. However, stemming does not guarantee higher classification accuracy because it can also merge words with different meanings. The best pipeline should be selected by comparing actual experimental results.